

Gaming Platform Backend

Debug existing classes, store match results, and produce player statistics.

Status: An original practice reconstruction based on your three-task notes. It is not a verified or official Karat question. The original starter code, bug, method signatures, and detailed data model were not supplied.

Scenario

You are given an in-memory backend for a gaming platform. Players register before playing. Completed matches record both participants, their scores, an outcome, and a timestamp. Extend the supplied classes without changing their public interfaces.

Provided class	Responsibility / fields for this mock
Player	A registered player: <code>player_id</code> , <code>name</code> , <code>stats</code> .
MatchResult	One match: <code>player1_id</code> , <code>player2_id</code> , <code>player1_score</code> , <code>player2_score</code> , <code>outcome</code> , <code>timestamp</code> .
PlayerStats	One player's history: <code>player_id</code> , <code>match_results</code> .
HeadToHead	A pair summary: IDs, wins by each player, draws, <code>total_matches</code> , <code>last_result</code> , <code>last_match_timestamp</code> .
GameManager	Player registry, registration, match storage, and statistics queries.

Outcome convention

The supporting enum has WIN, LOSS, and DRAW. A stored `MatchResult.outcome` is from that record's `player1_id` perspective. The supplied `outcome_for(player_id)` and `score_for(player_id)` helpers select the correct outcome and score for either participant.

For example, a record saying B beat A is a WIN for B and a LOSS for A, regardless of which player is passed first in a statistics query.

Scope and inputs

Use Python 3.10+. No external packages, database, API, or matchmaking are required. Scores are valid nonnegative integers. Outcomes are authoritative. Player IDs in a match or pair query are distinct. Timestamps are comparable UTC datetime values. Each actual match is submitted once; duplicate detection is outside this exercise.

Suggested practice budget: 45-60 minutes. This is a practice suggestion, not a verified interview time limit.

Tasks 1 and 2

Task 1 - Debug the existing implementation

Read the supplied classes and the independent-histories test. Identify why it fails, explain the cause, and fix the implementation. Keep support for an explicit initial history. Do not change the test's expected behavior.

```
first = PlayerStats("A")
other = PlayerStats("C")
result = MatchResult(
    "A", "B", 90, 60, Outcome.WIN,
    datetime.fromisoformat("2026-10-01T00:00:00+00:00"),
)
first.match_results.append(result)
assert len(first.match_results) == 1
assert other.match_results == [] # This assertion currently fails.
```

The bug supplied in this mock is representative; the notes do not identify the original bug.

Task 2a - add_match_result(result)

Implement `GameManager.add_match_result(result)`. For this whole-match data model:

1. Check that both participants are registered.
2. If either is missing, ignore the entire result without raising or changing either history.
3. Otherwise, store the match once in each participant's own history.
4. Return `None`. Do not register a player automatically.

Task 2b - get_average_score_by_outcome(player_id)

Implement `GameManager.get_average_score_by_outcome(player_id)`. Return a dictionary mapping Outcome enum members to the arithmetic mean of this player's own scores for that outcome.

Calculate WIN, LOSS, and DRAW separately. Omit outcomes with no matches. Return `{}` for an empty history or an unknown player. Do not round the averages. Dictionary order does not matter.

```
# A player has WIN scores 90 and 80, and one DRAW score of 70.
# With no losses, the expected result is:
{Outcome.WIN: 85.0, Outcome.DRAW: 70.0}
```

Requirements to preserve

One player's history must not pick up unrelated matches. A player may appear in either position in a match record. Score 0 is a valid score. Querying statistics must not change stored history.

Task 3 - Head-to-head statistics

Implement `GameManager.get_head_to_head(player1_id, player2_id)`. Return a `HeadToHead` object summarizing only matches between these two players. Include records with the participants in either order. Count each actual match once.

Field	Required meaning
<code>player1_wins / player2_wins</code>	Wins by the first / second player passed to the query.
<code>draws / total_matches</code>	Number of draws / number of matches between this pair.
<code>last_result</code>	WIN, LOSS, or DRAW from the first queried player's perspective.
<code>last_match_timestamp</code>	Maximum timestamp among this pair's matches.

If the pair has never played, or either player is unknown, return zero for all counts and `None` for both last fields. Match insertion order may differ from timestamp order. If the latest timestamps tie, either tied match may supply `last_result`; the timestamp must still be the maximum.

Shared example

Register A (Alice), B (Bob), and C (Cara). Insert these records in the order R2, R1, R3, R4. Dates below represent UTC midnight.

Record	Players	Scores	Stored outcome	Timestamp
R1	A / B	90 / 60	WIN for A	2026-10-01
R2	A / B	70 / 70	DRAW	2026-10-03
R3	B / A	80 / 50	WIN for B	2026-10-02
R4	A / C	80 / 40	WIN for A	2026-10-04

```
get_average_score_by_outcome("A")
# {Outcome.WIN: 85.0, Outcome.LOSS: 50.0, Outcome.DRAW: 70.0}

get_average_score_by_outcome("C")
# {Outcome.LOSS: 40.0}

get_head_to_head("A", "B")
# player1_wins=1, player2_wins=1, draws=1, total_matches=3
# last_result=Outcome.DRAW, last_match_timestamp=2026-10-03 UTC

get_head_to_head("B", "C")
# Both wins=0, draws=0, total_matches=0
# last_result=None, last_match_timestamp=None
```

Assumptions added during reconstruction

The original notes leave the record model, method ownership, unknown-query behavior, tie handling, and `last_result` type open. This mock uses whole-match records, requires both players to be registered, and returns an `Outcome` for `last_result`. The original could instead store per-player records or return the complete latest `MatchResult`.

Starter code - provided classes

game_platform.py / Part 1 of 2

```
from dataclasses import dataclass
from datetime import datetime
from enum import Enum
```

```
class Outcome(Enum):
    WIN = "WIN"
    LOSS = "LOSS"
    DRAW = "DRAW"
```

```
@dataclass(frozen=True)
class MatchResult:
    player1_id: str
    player2_id: str
    player1_score: int
    player2_score: int
    outcome: Outcome # From player1_id's perspective.
    timestamp: datetime

    def outcome_for(self, player_id: str) -> Outcome:
        if player_id == self.player1_id:
            return self.outcome
        if player_id == self.player2_id:
            if self.outcome == Outcome.DRAW:
                return Outcome.DRAW
            return Outcome.LOSS if self.outcome == Outcome.WIN else Outcome.WIN
        raise ValueError("Player did not participate in this match")

    def score_for(self, player_id: str) -> int:
        if player_id == self.player1_id:
            return self.player1_score
        if player_id == self.player2_id:
            return self.player2_score
        raise ValueError("Player did not participate in this match")
```

```
class PlayerStats:
    def __init__(self, player_id: str, match_results=[]):
        self.player_id = player_id
        self.match_results = match_results
```

```
class Player:
    def __init__(self, player_id: str, name: str):
        self.player_id = player_id
        self.name = name
        self.stats = PlayerStats(player_id)
```

```
@dataclass
class HeadToHead:
    player1_id: str
    player2_id: str
    player1_wins: int = 0
    player2_wins: int = 0
    draws: int = 0
    total_matches: int = 0
    last_result: Outcome | None = None
    last_match_timestamp: datetime | None = None
```

Starter code - GameManager

game_platform.py / Part 2 of 2

```
class GameManager:
    def __init__(self):
        self.players: dict[str, Player] = {}

    def register_player(self, player: Player) -> None:
        if player.player_id not in self.players:
            self.players[player.player_id] = player

    def get_player_stats(self, player_id: str) -> PlayerStats | None:
        player = self.players.get(player_id)
        return player.stats if player is not None else None

    def add_match_result(self, result: MatchResult) -> None:
        """Task 2a: store a valid result in both participants' histories.

        Ignore the entire result if either participant is unregistered.
        Assume callers submit each match once, with two distinct player IDs.
        """
        raise NotImplementedError("Task 2a")

    def get_average_score_by_outcome(
        self, player_id: str
    ) -> dict[Outcome, float]:
        """Task 2b: group this player's own scores by their own outcomes.

        Omit empty groups; return {} for an unknown player or empty history.
        Do not round the calculated averages.
        """
        raise NotImplementedError("Task 2b")

    def get_head_to_head(
        self, player1_id: str, player2_id: str
    ) -> HeadToHead:
        """Task 3: summarize only the matches between these two players.

        last_result is the latest outcome from the first argument's perspective.
        Find the latest match by timestamp, not by insertion order.
        Count each match once. Return zero counts and None values for no history
        or an unknown player. For equal latest timestamps, either tied match
        may supply last_result; the timestamp must be the maximum timestamp.
        """
        raise NotImplementedError("Task 3")
```

Run the public tests incrementally

```
python -m unittest -v test_game_platform.Task1
python -m unittest -v test_game_platform.Task2Add
python -m unittest -v test_game_platform.Task2Average
python -m unittest -v test_game_platform.Task3
python -m unittest -v test_game_platform
```

The accompanying ZIP contains the editable starter, 15 public tests, and setup instructions. Initially the Task 1 isolation assertion fails and the three unimplemented methods raise `NotImplementedError`. No solution is included.

When you explain your implementation, cover outcome perspective, empty history, unknown players, reversed participants, unrelated opponents, timestamp ordering, and time complexity.